

Math 7243 Machine Learning - Homework 3

Wilson Glass

For programming questions, you can only use numpy library. You should not use any build in function from Scikit-learn or StatsModels libraries.

Problem 1 - Gradient Decent and Newton's method

Consider solving the problem of locally weighted linear regression using gradient descent and Newton's method. Given data $\{\vec{x}^{(i)}, y^{(i)}\}$ for $i = 1, 2, \dots, n$ and a query point $\vec{x}$, we choose a parameter vector θ to minimize the loss function

$$J(\vec{\theta}; \vec{x}) = \sum_{i=1}^n w^{(i)} (\vec{\theta}^T \vec{x}^{(i)} - y^{(i)})^2$$

Here the weight function is $w^{(i)} = \exp\left(-\frac{\|\vec{x}^{(i)} - \vec{x}\|^2}{2\tau^2}\right)$ where τ is a hyper-parameter that must be tuned. Note that whenever we receive a new query point $\vec{x}$, we must solve the entire problem again with these new weights $w^{(i)}$.

(a) Given a data point $\vec{x}$, derive the gradient of $J(\vec{\theta}; \vec{x})$ with respect to $\vec{\theta}$.

Solution. The weights $w^{(i)}$ depend only on the query point $\vec{x}$, the data and τ , not on $\vec{\theta}$, so they are constants when differentiating with respect to $\vec{\theta}$. Since $\nabla_{\vec{\theta}}(\vec{\theta}^T \vec{x}^{(i)} - y^{(i)}) = \vec{x}^{(i)}$, the chain rule applied to each term of the sum gives

$$\nabla_{\vec{\theta}} J(\vec{\theta}; \vec{x}) = 2 \sum_{i=1}^n w^{(i)} (\vec{\theta}^T \vec{x}^{(i)} - y^{(i)}) \vec{x}^{(i)}.$$

In matrix form, let $X \in \mathbb{R}^{n \times d}$ be the matrix whose i -th row is $(\vec{x}^{(i)})^T$, let $\vec{y} = (y^{(1)}, \dots, y^{(n)})^T$, and let $W = \text{diag}(w^{(1)}, \dots, w^{(n)})$. Then $J = (X\vec{\theta} - \vec{y})^T W (X\vec{\theta} - \vec{y})$ and

$$\nabla_{\vec{\theta}} J(\vec{\theta}; \vec{x}) = 2X^T W (X\vec{\theta} - \vec{y}).$$

(b) Given a data point $\vec{x}$, derive the Hessian of $J(\vec{\theta}; \vec{x})$ with respect to $\vec{\theta}$.

Solution. Differentiate the gradient from (a) once more. The only dependence on $\vec{\theta}$ is through the scalar $\vec{\theta}^T \vec{x}^{(i)}$, whose derivative with respect to $\vec{\theta}$ is $(\vec{x}^{(i)})^T$. Therefore

$$H = \nabla_{\vec{\theta}}^2 J(\vec{\theta}; \vec{x}) = 2 \sum_{i=1}^n w^{(i)} \vec{x}^{(i)} (\vec{x}^{(i)})^T = 2X^T W X.$$

H is a $d \times d$ matrix that does not depend on $\vec{\theta}$, because J is quadratic in $\vec{\theta}$. For any vector $\vec{v}$ we have $\vec{v}^T H \vec{v} = 2 \sum_i w^{(i)} (\vec{v}^T \vec{x}^{(i)})^2 \geq 0$ since every $w^{(i)} > 0$, so H is positive semidefinite and J is convex.

(c) Given a data point $\vec{x}$, write the update formula for gradient descent. Use the symbol η for an arbitrary step size.

Solution. Gradient descent steps against the gradient, $\vec{\theta}_{t+1} = \vec{\theta}_t - \eta \nabla_{\vec{\theta}} J(\vec{\theta}_t; \vec{x})$. Substituting the gradient from (a),

$$\vec{\theta}_{t+1} = \vec{\theta}_t - 2\eta \sum_{i=1}^n w^{(i)} (\vec{\theta}_t^T \vec{x}^{(i)} - y^{(i)}) \vec{x}^{(i)} = \vec{\theta}_t - 2\eta X^T W (X\vec{\theta}_t - \vec{y}).$$

The weights $w^{(i)}$ are computed once for the given query point $\vec{x}$ and stay fixed over the iterations; only $\vec{\theta}$ changes.

(d) Given a data point $\vec{x}$, write the update formula for Newton's method.

Solution. Newton's method replaces the step size by the inverse Hessian, $\vec{\theta}_{t+1} = \vec{\theta}_t - H^{-1} \nabla_{\vec{\theta}} J(\vec{\theta}_t; \vec{x})$. Using (a) and (b), and assuming $X^T W X$ is invertible,

$$\vec{\theta}_{t+1} = \vec{\theta}_t - \left(2 \sum_{i=1}^n w^{(i)} \vec{x}^{(i)} (\vec{x}^{(i)})^T\right)^{-1} \left(2 \sum_{i=1}^n w^{(i)} (\vec{\theta}_t^T \vec{x}^{(i)} - y^{(i)}) \vec{x}^{(i)}\right) = \vec{\theta}_t - (X^T W X)^{-1} X^T W (X\vec{\theta}_t - \vec{y}).$$

The factors of 2 cancel. Expanding the product, $(X^T W X)^{-1} X^T W X \vec{\theta}_t = \vec{\theta}_t$, so the $\vec{\theta}_t$ terms cancel as well and

$$\vec{\theta}_{t+1} = (X^T W X)^{-1} X^T W \vec{y} = \left(\sum_{i=1}^n w^{(i)} \vec{x}^{(i)} (\vec{x}^{(i)})^T \right)^{-1} \sum_{i=1}^n w^{(i)} y^{(i)} \vec{x}^{(i)}.$$

This is the closed-form weighted least squares solution. Because J is exactly quadratic, Newton's method reaches the minimizer in a single step from any starting point.

Problem 2 - (Stochastic) Gradient Decent

(1) The data file $\{\vec{x}^{(i)}, y^{(i)}\}$ for $i = 1, 2, \dots, n$ is drawn (with noise) from

$$f(x) = \beta_0 + \beta_1 \sin(x) + \beta_2 \cos(x)$$

Can you solve the parameters use the least squares method? Find a closed formula and explain the matrices clearly in your formula.

Solution. Yes. The model is not linear in x , but it is linear in the parameters $\beta_0, \beta_1, \beta_2$, which is all that least squares requires. For each data point, $\sin(x^{(i)})$ and $\cos(x^{(i)})$ are known numbers, so they can be used as features. Define

$$X = \begin{pmatrix} 1 & \sin(x^{(1)}) & \cos(x^{(1)}) \\ 1 & \sin(x^{(2)}) & \cos(x^{(2)}) \\ \vdots & \vdots & \vdots \\ 1 & \sin(x^{(n)}) & \cos(x^{(n)}) \end{pmatrix} \in \mathbb{R}^{n \times 3}, \quad \vec{y} = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{pmatrix} \in \mathbb{R}^n, \quad \vec{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix} \in \mathbb{R}^3.$$

X is the design matrix with one row per data point: the first column is all ones (for the intercept β_0), the second column holds $\sin(x^{(i)})$ (for β_1) and the third holds $\cos(x^{(i)})$ (for β_2). $\vec{y}$ is the vector of observed outputs and $\vec{\beta}$ is the vector of unknown parameters. The model is then $\vec{y} \approx X\vec{\beta}$ and the loss is

$$RSS(\vec{\beta}) = \|\vec{y} - X\vec{\beta}\|^2 = (\vec{y} - X\vec{\beta})^T (\vec{y} - X\vec{\beta}).$$

Setting the gradient to zero, $\nabla_{\vec{\beta}} RSS = -2X^T(\vec{y} - X\vec{\beta}) = 0$, gives the normal equations $X^T X \vec{\beta} = X^T \vec{y}$, and therefore

$$\hat{\vec{\beta}} = (X^T X)^{-1} X^T \vec{y}.$$

Here $X^T X$ is a 3×3 matrix. It is invertible when the three columns of X are linearly independent, and since the Hessian $2X^T X$ is then positive definite, $\hat{\vec{\beta}}$ is the unique minimizer.

(2) The data file $\{\vec{x}^{(i)}, y^{(i)}\}$ for $i = 1, 2, \dots, n = 10$ is drawn (with noise) from the function:

$$g(x) = \beta_0 + \sin(\beta_1 x) + \cos(\beta_2 x)$$

$x^{(i)}$	0	2	4	6	8	10	12	14	16	18
$y^{(i)}$	2.85	1.5	0.49	1.57	1.9	0.6	0.38	2.33	1.65	0.3

Use gradient decent(GD) or stochastic gradient decent (SGD) to fit the data to the function $g(x)$ by minimizing the RSS loss

$$RSS = \sum_{i=1}^n (y^{(i)} - g(x^{(i)}))^2$$

Turn in any associated computations, your learning rate, and the parameters.

Solution. Here β_1 and β_2 sit inside the sine and cosine, so the model is not linear in the parameters and the closed formula from part (1) does not apply. I used gradient descent.

Gradient. Write the residual as $r^{(i)} = y^{(i)} - \beta_0 - \sin(\beta_1 x^{(i)}) - \cos(\beta_2 x^{(i)})$, so that $RSS = \sum_i (r^{(i)})^2$. By the chain rule,

$$\frac{\partial RSS}{\partial \beta_0} = -2 \sum_{i=1}^n r^{(i)}, \quad \frac{\partial RSS}{\partial \beta_1} = -2 \sum_{i=1}^n r^{(i)} x^{(i)} \cos(\beta_1 x^{(i)}), \quad \frac{\partial RSS}{\partial \beta_2} = 2 \sum_{i=1}^n r^{(i)} x^{(i)} \sin(\beta_2 x^{(i)}).$$

I checked these against central finite differences; the largest difference was about 7×10^{-9} .

Update. With $\vec{\beta} = (\beta_0, \beta_1, \beta_2)^T$, each iteration updates all three parameters at once:

$$\vec{\beta}_{t+1} = \vec{\beta}_t - \eta \nabla_{\vec{\beta}} RSS(\vec{\beta}_t).$$

Starting point. The RSS is not convex in (β_1, β_2) , so gradient descent converges to the local minimum nearest its starting point. To choose a good start I evaluated the RSS on a grid of (β_1, β_2) values with spacing 0.02, setting β_0 to its best value for each pair, which is the mean of $y^{(i)} - \sin(\beta_1 x^{(i)}) - \cos(\beta_2 x^{(i)})$. Because every $x^{(i)}$ is an even integer, changing β_1 or β_2 by π leaves all ten predictions unchanged, so the grid only needs to cover $\beta_1 \in [-\pi/2, \pi/2]$ and $\beta_2 \in [0, \pi/2]$. The best grid point, rounded to two decimals, was $(\beta_0, \beta_1, \beta_2) = (2.02, -0.14, 0.86)$ with $RSS = 0.2873$.

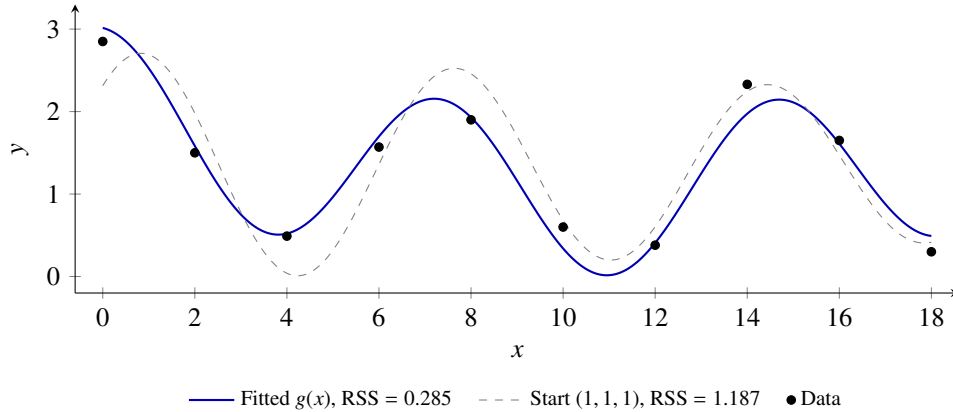
Learning rate and result. I used $\eta = 10^{-4}$ for 100,000 iterations.

Iteration	β_0	β_1	β_2	RSS
0	2.0200	-0.1400	0.8600	0.2873
10	2.0200	-0.1410	0.8604	0.2862
100	2.0192	-0.1423	0.8609	0.2856
1,000	2.0156	-0.1425	0.8609	0.2854
5,000	2.0148	-0.1425	0.8609	0.2854
100,000	2.0148	-0.1425	0.8609	0.2854

The fitted parameters are

$$\beta_0 \approx 2.015, \quad \beta_1 \approx -0.143, \quad \beta_2 \approx 0.861, \quad RSS \approx 0.285.$$

The norm of the gradient at this point is about 2×10^{-12} , so the iteration has converged. Stochastic gradient descent from the same start (one data point per update, step size $10^{-3}/(1 + \text{epoch}/2000)$, 20,000 epochs) gives $(2.015, -0.142, 0.861)$ with the same RSS to four decimals.



$x^{(i)}$	0	2	4	6	8	10	12	14	16	18
$y^{(i)}$	2.85	1.50	0.49	1.57	1.90	0.60	0.38	2.33	1.65	0.30
$g(x^{(i)})$	3.015	1.583	0.520	1.698	1.929	0.340	0.408	1.974	1.611	0.492
residual	-0.165	-0.083	-0.030	-0.128	-0.029	0.260	-0.028	0.356	0.039	-0.192

Remarks. (i) The result depends on the starting point. For example, starting from $(1, 1, 1)$ with the same learning rate, gradient descent converges to $(1.314, 0.905, 0.940)$ with $RSS = 1.187$, a local minimum with a worse fit (dashed curve). (ii) The parameters are not unique: since every $x^{(i)}$ is an even integer, $\beta_1 \approx -0.143 + \pi \approx 2.999$ gives exactly the same predictions on the data, and likewise for shifts of β_2 . (iii) From this start $\eta = 10^{-3}$ also converges to the same parameters, while $\eta = 2 \times 10^{-3}$ does not, so $\eta = 10^{-4}$ is a safe choice.

Code (numpy only).

```
import numpy as np

# Data
x = np.arange(0, 20, 2, dtype=float)
y = np.array([2.85, 1.5, 0.49, 1.57, 1.9, 0.6, 0.38, 2.33, 1.65, 0.3])

def g(b, x):
    return b[0] + np.sin(b[1] * x) + np.cos(b[2] * x)

def rss(b):
    return np.sum((y - g(b, x)) ** 2)

def grad(b, xs=x, ys=y):
    """Gradient of RSS with respect to (b0, b1, b2)."""
    r = ys - g(b, xs) # residuals
    return np.array([
        -2 * np.sum(r), # d/d b0
        -2 * np.sum(r * xs * np.cos(b[1] * xs)), # d/d b1
        2 * np.sum(r * xs * np.sin(b[2] * xs)), # d/d b2
    ])

def gradient_descent(b_init, lr=1e-4, iters=100_000, report=()):
    b = np.array(b_init, dtype=float)
    for t in range(1, iters + 1):
        b = b - lr * grad(b)
        if t in report:
            print(f" iter {t:6d} beta = {np.round(b, 4)} RSS = {rss(b):.4f}")
    return b

def sgd(b_init, lr=1e-3, epochs=20_000, seed=0):
    """One data point per update, with a decaying step size."""
    rng = np.random.default_rng(seed)
    b = np.array(b_init, dtype=float)
    for epoch in range(epochs):
        step = lr / (1 + epoch / 2000)
        for i in rng.permutation(len(x)):
            b = b - step * grad(b, x[i:i+1], y[i:i+1])
    return b

if __name__ == "__main__":
    # Check the analytic gradient against central finite differences
    b_test, eps = np.array([0.7, 1.3, 0.4]), 1e-6
    num = np.array([(rss(b_test + eps * e) - rss(b_test - eps * e)) / (2 * eps)
                    for e in np.eye(3)])
    print("gradient check, max abs error:", np.max(np.abs(num - grad(b_test))))

    # RSS is not convex in (b1, b2), so GD finds the minimum nearest its start.
    # Choose the start with a grid search: for each (b1, b2) the best b0 is the
    # mean of y - sin(b1 x) - cos(b2 x). Every x is an even integer, so shifting
    # b1 or b2 by pi does not change the predictions; b1 in [-pi/2, pi/2] and
    # b2 in [0, pi/2] therefore covers every case.
    best = None
    for b1 in np.arange(-1.56, 1.57, 0.02):
        for b2 in np.arange(0.0, 1.57, 0.02):
            b0 = np.mean(y - np.sin(b1 * x) - np.cos(b2 * x))
            cand = np.array([b0, b1, b2])
            if best is None or rss(cand) < rss(best):
                best = cand
    init = np.round(best, 2)
    print("grid starting point:", init, " RSS there:", round(rss(init), 4))

    print("gradient descent, lr = 1e-4:")
    b_gd = gradient_descent(init, lr=1e-4, iters=100_000,
                            report=(1, 10, 100, 1000, 5000, 100_000))
    print("GD result:", np.round(b_gd, 4), " RSS:", round(rss(b_gd), 4),
          " |grad|:", np.linalg.norm(grad(b_gd)))

    b_sgd = sgd(init)
    print("SGD result:", np.round(b_sgd, 4), " RSS:", round(rss(b_sgd), 4))

    b_l1l = gradient_descent([1.0, 1.0, 1.0], lr=1e-4, iters=100_000)
    print("GD from (1, 1, 1):", np.round(b_l1l, 4), " RSS:", round(rss(b_l1l), 4))

    print("fitted values:")
    for xi, yi, fi in zip(x, y, g(b_gd, x)):
        print(f" x = {xi:2.0f} y = {yi:4.2f} g = {fi:5.3f} residual = {yi - fi:+.3f}")
```